

# Chocolate Distribution Problem

## Problem Statement

Given an array `arr[]` of positive integers, where each element represents the number of chocolates in a packet, and an integer `m` representing the number of students, distribute exactly one packet to each student such that the difference between the maximum and minimum number of chocolates received is minimized.

Return the minimum possible difference.

---

## Example 1

### Input

```
arr = [3, 4, 1, 9, 56, 7, 9, 12]
m = 5
```

### Output

```
6
```

### Explanation

After sorting:

```
[1, 3, 4, 7, 9, 9, 12, 56]
```

Consider groups of `5` consecutive packets:

```
[1, 3, 4, 7, 9]    → 9 - 1 = 8
[3, 4, 7, 9, 9]    → 9 - 3 = 6
[4, 7, 9, 9, 12]   → 12 - 4 = 8
[7, 9, 9, 12, 56]  → 56 - 7 = 49
```

The minimum difference is:

6

## Example 2

### Input

```
arr = [7, 3, 2, 4, 9, 12, 56]
m = 3
```

### Output

2

### Explanation

After sorting:

[2, 3, 4, 7, 9, 12, 56]

Consider groups of 3 packets:

|             |               |
|-------------|---------------|
| [2, 3, 4]   | → 4 - 2 = 2   |
| [3, 4, 7]   | → 7 - 3 = 4   |
| [4, 7, 9]   | → 9 - 4 = 5   |
| [7, 9, 12]  | → 12 - 7 = 5  |
| [9, 12, 56] | → 56 - 9 = 47 |

The minimum difference is:

2

## Approach

The optimal approach is:

1. Sort the array.

2. Consider every group of `m` consecutive elements.
3. For each group, calculate:

```
maximum - minimum
```

1. Keep track of the minimum difference.

### Why does sorting help?

After sorting, the minimum and maximum values of any group of `m` consecutive elements are at the two ends of that group.

For a starting index `i`:

```
Minimum = arr[i]  
Maximum = arr[i + m - 1]
```

Therefore:

```
difference = arr[i + m - 1] - arr[i]
```

---

## Java Implementation

```
import java.util.Arrays;  
  
class Solution {  
    public int findMinDiff(int arr[], int m) {  
  
        Arrays.sort(arr);  
  
        int min = Integer.MAX_VALUE;  
  
        for (int i = 0; i <= arr.length - m; i++) {  
  
            int dif = arr[m + i - 1] - arr[i];  
  
            min = Math.min(dif, min);  
        }  
  
        return min;  
    }  
}
```

```
}  
}
```

---

## Code Explanation

### 1. Sort the array

```
Arrays.sort(arr);
```

This arranges the packets in increasing order.

Example:

```
[3, 4, 1, 9, 56, 7, 9, 12]
```

becomes:

```
[1, 3, 4, 7, 9, 9, 12, 56]
```

---

### 2. Initialize minimum difference

```
int min = Integer.MAX_VALUE;
```

We initially assume that the minimum difference is very large.

---

### 3. Iterate through possible groups

```
for (int i = 0; i <= arr.length - m; i++)
```

Here, `i` represents the starting index of the group.

The condition:

```
i <= arr.length - m
```

ensures that every possible group containing exactly `m` packets is checked.

---

#### 4. Calculate the difference

```
int dif = arr[m + i - 1] - arr[i];
```

The last element of the current group is:

```
arr[i + m - 1]
```

The first element is:

```
arr[i]
```

Therefore:

```
difference = arr[i + m - 1] - arr[i]
```

---

#### 5. Update the minimum

```
min = Math.min(dif, min);
```

If the current difference is smaller than the previously found difference, update `min`.

---

#### 6. Return the answer

```
return min;
```

After checking all possible groups, `min` contains the minimum possible difference.

---

## Dry Run

Input:

```
arr = [3, 4, 1, 9, 56, 7, 9, 12]
m = 5
```

After sorting:

```
[1, 3, 4, 7, 9, 9, 12, 56]
```

| i | Selected Packets  | Difference    | min |
|---|-------------------|---------------|-----|
| 0 | [1, 3, 4, 7, 9]   | $9 - 1 = 8$   | 8   |
| 1 | [3, 4, 7, 9, 9]   | $9 - 3 = 6$   | 6   |
| 2 | [4, 7, 9, 9, 12]  | $12 - 4 = 8$  | 6   |
| 3 | [7, 9, 9, 12, 56] | $56 - 7 = 49$ | 6   |

Final answer:

6

## Complexity Analysis

### Time Complexity

Sorting takes:

$O(n \log n)$

The loop takes:

$O(n)$

Therefore:

Overall Time Complexity =  $O(n \log n)$

## Space Complexity

The algorithm uses only a few extra variables:

```
Space Complexity =  $O(1)$ 
```

apart from the space used internally by the sorting implementation.

---

## Key DSA Concept

This problem uses:

- **Sorting**
- **Sliding Window**
- **Greedy observation**

## Important Pattern

Whenever you need to select `m` elements such that:

```
maximum - minimum
```

is minimized, consider:

1. Sort the array
2. Check windows of size `m`
3. Calculate `arr[i + m - 1] - arr[i]`
4. Take the minimum

## Edge Cases

```
m = 1
```

If only one student exists, one packet is selected.

```
maximum - minimum = 0
```

```
m = arr.length
```

All packets must be distributed.

Therefore:

```
answer = maximum element - minimum element
```

## Duplicate values

If duplicate packet sizes exist, the difference can be:

```
0
```

For example:

```
arr = [5, 5, 5]  
m = 2
```

Answer:

```
5 - 5 = 0
```

---

## Final Complexity

```
Time Complexity : O(n log n)  
Space Complexity : O(1) auxiliary
```

**Pattern:** Sort → Fixed-size Window → Minimum Difference